

présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

*Dans le cadre du Stage d'été de la
2^e année Génie des Télécommunications*

élaboré par

Mohamed OMEZZINE

**AUTOMATISATION DU TRAITEMENT DES TICKETS ET
DES APPELS ENTRANTS PAR L'INTELLIGENCE
ARTIFICIELLE**

Mlle. Chayma MAHWACHI
M. Mohamed OTHMEN

Encadrant Entreprise
Responsable Entreprise



REMERCIEMENTS

Au terme de ce stage d'été, je tiens à exprimer ma profonde gratitude envers toutes les personnes dont le soutien et les conseils ont été déterminants dans la réussite de ce travail.

Je remercie avant tout ALLAH, le Tout-Puissant, de m'avoir accordé la force, la santé et la volonté nécessaires pour mener à bien cette expérience.

J'adresse mes plus sincères remerciements à mon encadrante professionnelle, **Mlle Chayma MAHWACHI**, pour son encadrement rigoureux, ses conseils avisés et sa disponibilité constante. Son expertise technique et son accompagnement tout au long de ce stage ont grandement enrichi mon parcours.

Mes remerciements vont également à **Monsieur Mohamed OTHMEN**, fondateur de CYBERSHIELD TN, pour son accueil au sein de l'entreprise et la confiance qu'il m'a accordée en me donnant l'opportunité d'évoluer dans un environnement stimulant.

Enfin, je tiens à exprimer ma reconnaissance aux membres du jury qui font l'honneur d'évaluer ce rapport de stage. Leurs remarques et suggestions constructives seront d'une grande valeur pour la suite de mon parcours.



TABLE DES MATIÈRES

Remerciements	i
LISTE DES FIGURES	v
LISTE DES ABRÉVIATIONS	vi
INTRODUCTION GÉNÉRALE	1
1 Contexte générale du projet	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.2.1 CYBERSHIELD TN	3
1.2.2 Services Offertes	4
1.3 Cadre générale de projet	4
1.3.1 Etude de l'existant	4
1.3.2 Critique de l'existant	4
1.3.3 Solution envisagée	5
1.4 Spécification des besoins	5
1.4.1 Besoins fonctionnels	5
1.4.2 Besoins non fonctionnels	6
1.5 Plannification de projet	6
1.6 Conclusion	6
2 Conception de la solution	7
2.1 Introduction	7
2.2 Architecture globale de la solution	7
2.2.1 Vue d'ensemble et démarche itérative	7
2.2.2 Pipeline de notification, de triage et d'escalade des tickets (Telegram – Ollama – Zammad)	7

2.2.3	Pipeline d'appel en direct (3CX – Asterisk – bridge RTP – STT – bot – TTS)	8
2.3	Choix technologiques	8
2.3.1	n8n (orchestration des workflows)	8
2.3.2	Ollama (LLM local)	9
2.3.3	Redis (file d'attente et gestion de la concurrence)	9
2.3.4	Zammad (gestion des tickets)	10
2.3.5	Telegram (notification et validation par un agent)	11
2.3.6	faster-whisper (reconnaissance vocale)	11
2.3.7	edge-tts (synthèse vocale)	11
2.4	Conclusion	12
3	Réalisation et tests	13
3.1	Introduction	13
3.2	Environnement de travail	13
3.2.1	Outils et technologies (Docker, WSL2, VMware, ngrok)	13
3.3	Orchestration des notifications et validation des réponses via Telegram	13
3.3.1	Identification de l'agent et notification initiale	14
3.3.2	Sondage, relance et gestion de la disponibilité	15
3.3.3	Génération automatique de propositions de solution par l'IA (Ollama)	17
3.3.4	Validation de la réponse et mécanisme <i>Force Reply</i>	18
3.4	Mise en œuvre du pipeline d'appel en direct	19
3.4.1	Transfert de l'appel du client vers l'agent IA	19
3.4.2	Script de collecte d'informations (Live Support Bot - Decide)	21
3.4.3	Boucle jusqu'à la fin de l'appel et création du ticket	23
3.4.4	Workflow n8n « Call to Ticket »	23
3.5	Difficultés rencontrées et solutions	25
3.5.1	Problèmes réseau entre environnements distribués	25
3.5.2	Fiabilité de la passerelle bridge.py	26
3.5.3	Erreur de validation du workflow n8n	27
3.5.4	Latence perçue par l'appelant	28
3.5.5	Limites du modèle de reconnaissance vocale	28
3.6	Conclusion	28
	CONCLUSION GÉNÉRALE	30
	BIBLIOGRAPHIE	31

LISTE DES FIGURES

1.1	Logo de CYBERSHIELD TN	4
1.2	Architecture cible de la solution	5
2.1	Pipeline de génération assistée et de réponse aux tickets	8
2.2	Extrait du workflow n8n du pipeline d'appel en direct	9
2.3	Configuration Docker Compose de l'architecture n8n en mode queue avec Redis	10
2.4	Configuration de l'appel HTTP vers le service faster-whisper depuis n8n	11
2.5	Sortie du nœud Edge TTS - Generate Speech, montrant l'audio MP3 généré à partir du texte du bot	12
3.1	Résolution de l'agent et notification du groupe lors de la prise en charge du ticket	14
3.2	Notification Telegram reçue lors de la création d'un ticket	15
3.3	Prise en charge du ticket par un agent	15
3.4	Boucle de sondage Telegram en attente d'une réponse de l'agent	16
3.5	Mécanisme de relance unique et de réassignation en cas d'absence de réponse .	17
3.6	Workflow n8n de génération et de validation de la solution IA	18
3.7	Sollicitation de l'assistance IA par l'agent, puis proposition de solution générée par Ollama	18
3.8	Réponse de l'agent rattachée au ticket via le mécanisme Force Reply	19
3.9	Workflow n8n gérant l'attente et la validation d'une réponse manuscrite via Force Reply	19
3.10	Règle de renvoi configurée dans 3CX (extension 100 vers l'extension IA via le préfixe 9101)	20
3.11	Journal de bridge.py confirmant l'acceptation de la connexion AudioSocket et l'envoi du webhook Call Started	20
3.12	Historique d'appels 3CX confirmant le routage effectif vers l'extension IA . . .	21
3.13	Détection des réponses de confirmation et de refus du client	21
3.14	Machine à états du script de collecte, avec validation des réponses via isGarbage()	22
3.15	Exemple réel de champs collectés corrompus par une transcription défectueuse (nom et email vides, société mal transcrite)	22

3.16	Journal de <code>bridge.py</code> montrant un cycle complet de traitement d'un tour de parole	23
3.17	Ajout d'une entrée manuelle dans le fichier <code>hosts</code> pour forcer la résolution vers l'adresse IP interne du serveur 3CX	26
3.18	Extension 100 enregistrée avec succès après correction DNS, passant au statut « Available »	26
3.19	Erreur de validation n8n bloquant l'exécution du workflow suite à la présence de nœuds Respond to Webhook non utilisés	27



LISTE DES ABRÉVIATIONS

API Application Programming Interface

DID Direct Inward Dialing

DNS Domain Name System

DTMF Dual-Tone Multi-Frequency

FQDN Fully Qualified Domain Name

HTTP HyperText Transfer Protocol

IA Intelligence Artificielle

IP Internet Protocol

JSON JavaScript Object Notation

LAN Local Area Network

LLM Large Language Model

MP3 MPEG-1 Audio Layer 3

NAT Network Address Translation

PABX Private Automatic Branch Exchange

PCM Pulse Code Modulation

PJSIP Project on SIP (bibliothèque SIP open source)

REST Representational State Transfer

RMS Root Mean Square

RTP Real-time Transport Protocol

SIP Session Initiation Protocol

SOC Security Operations Center

SOCaaS Security Operations Center as a Service

STT Speech-to-Text

TCP Transmission Control Protocol

TTS Text-to-Speech

UUID Universally Unique Identifier

VAD Voice Activity Detection

VoIP Voice over Internet Protocol

VPN Virtual Private Network

WSL2 Windows Subsystem for Linux 2

MSSP Managed Security Service Provider

Zero Trust Modèle de sécurité reposant sur la vérification systématique de chaque accès, sans confiance implicite

N8N Nodemation



INTRODUCTION GÉNÉRALE

Dans de nombreuses entreprises, le traitement des appels et des tickets de support reste largement manuel, ce qui mobilise un temps considérable pour les équipes concernées et retarde la prise en charge des demandes les plus simples. Pour une entreprise spécialisée en cybersécurité comme CYBERSHIELD TN, cette charge est d'autant plus problématique qu'elle détourne les équipes du SOC de leur mission première : la supervision et la réponse aux incidents de sécurité.

C'est dans ce contexte que s'inscrit ce rapport, rédigé à l'issue d'un stage d'été de deux mois effectué au sein de CYBERSHIELD TN. Ce stage avait pour objectif de concevoir une solution capable d'assister les agents dans le traitement des tickets et des appels entrants, en s'appuyant sur des technologies d'intelligence artificielle telles que la reconnaissance vocale, la génération de langage et la synthèse vocale, tout en garantissant qu'aucune réponse générée automatiquement ne parvienne à un client sans validation préalable d'un agent.

Le travail a été mené en deux étapes complémentaires. La première a porté sur l'automatisation de la notification des tickets prioritaires vers Telegram, enrichie par la suite d'une proposition de solution générée par un modèle de langage et soumise systématiquement à l'approbation d'un agent. La seconde a consisté à étendre cette automatisation à l'appel téléphonique lui-même, à travers la conception d'un agent vocal capable de transcrire la parole de l'appelant, de mener une collecte structurée d'informations, et de créer automatiquement un ticket de support.

Ce rapport s'organise en trois chapitres. Le premier présente le contexte du stage, l'organisme d'accueil ainsi que la problématique et les besoins ayant motivé ce travail. Le deuxième expose la conception de la solution retenue, son architecture et les choix technologiques associés. Le

troisième décrit enfin sa réalisation concrète, les difficultés rencontrées au cours du développement, ainsi que les résultats obtenus à l'issue des tests menés.

Contexte générale du projet

1.1 Introduction

Ce rapport s'inscrit dans le cadre d'un stage d'été de deux mois effectué au sein de CYBERSHIELD TN, consacré à la conception d'une solution d'automatisation du traitement des appels entrants et des tickets de support client. Le constat de départ est simple : la gestion manuelle de ces demandes mobilise un temps considérable pour les équipes support, sans permettre de réponse rapide aux demandes simples et récurrentes.

Le travail s'est déroulé en deux étapes : d'abord l'automatisation de la notification des alertes Zammad vers Telegram (priorités *high*, *urgent*, *critical*), enrichie d'une proposition de solution par un modèle de langage et d'une validation obligatoire par un agent ; ensuite l'extension de cette automatisation à l'appel téléphonique lui-même, via un pipeline de reconnaissance vocale, de génération de réponse et de synthèse vocale intégré au standard téléphonique de l'entreprise.

1.2 Présentation de l'organisme d'accueil

1.2.1 CYBERSHIELD TN

CYBERSHIELD est un acteur de référence en cybersécurité, spécialisé dans des solutions de sécurité avancées destinées à ses partenaires et clients. L'entreprise combine technologies de pointe, analyse pilotée par l'IA et expertise approfondie pour protéger les organisations face aux menaces les plus sophistiquées, avec une posture de sécurité renforcée plutôt qu'une simple fourniture d'outils.



FIGURE 1.1 – Logo de CYBERSHIELD TN

1.2.2 Services Offertes

- **Consulting** : stratégie de cybersécurité selon l'approche Zero Trust.
- **Advisory** : gestion des risques, conformité et gouvernance.
- **SOCaaS** : supervision continue, en partenariat avec Seceon.

1.3 Cadre générale de projet

1.3.1 Etude de l'existant

3CX (téléphonie) et Zammad (tickets) étaient déjà en place, mais fonctionnent indépendamment : aucun lien automatique entre un appel et un ticket, le rapprochement devant être effectué manuellement par l'agent. Le traitement du contenu des demandes – écoute, compréhension, rédaction – repose entièrement sur l'intervention humaine, sans aucune automatisation ou assistance intelligente.

1.3.2 Critique de l'existant

- **Absence de lien automatisé** entre l'appel et le ticket, source d'erreurs et de charge répétitive.

- **Dépendance à la disponibilité humaine** pour tout traitement, ralentissant la prise en charge.
- **Absence de premier niveau de réponse automatisé**, même pour les demandes récurrentes.
- **Manque d’exploitation du contenu des échanges**, limitant traçabilité et analyse.
- **Charge additionnelle pour les équipes du SOC**, au détriment de la supervision de sécurité, mission première d’une entreprise de SOCaaS.

1.3.3 Solution envisagée

La solution consiste à interfacier une couche intelligente avec les outils existants (3CX, Zammad) plutôt qu’à les remplacer : interception des appels via 3CX/Asterisk, transcription (faster-whisper), génération de réponse par un LLM local (Ollama), restitution vocale (edge-tts), le tout orchestré par n8n. Cette automatisation vise à libérer du temps pour les équipes du SOC, au bénéfice de leur cœur de métier : la supervision de sécurité.

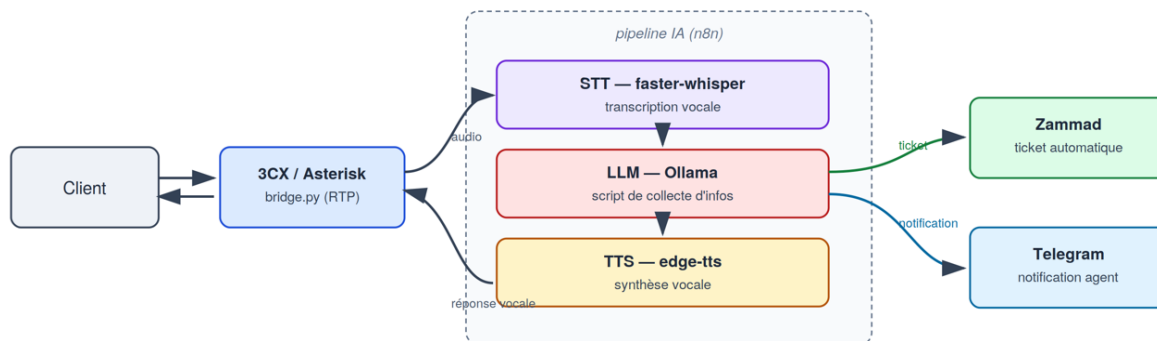


FIGURE 1.2 – Architecture cible de la solution

1.4 Spécification des besoins

1.4.1 Besoins fonctionnels

Détection automatique du début/fin d’appel, message d’accueil, transcription en temps réel, collecte structurée d’informations (nom, problème, email, société), détection et reformulation

en cas de réponse mal interprétée, restitution vocale sans coupure, création automatique de ticket, et clôture fiable de l'appel quelle que soit son issue.

1.4.2 Besoins non fonctionnels

Performance (latence perçue faible), fiabilité (résilience aux échecs ponctuels sans perte d'état), robustesse face aux erreurs de transcription, disponibilité de l'ensemble des services impliqués, maintenabilité (composants indépendants) et traçabilité de chaque étape du traitement.

1.5 Plannification de projet

Le stage (1^{er} juillet – 31 août 2026) s'est organisé en deux phases. La première a porté sur l'automatisation de la génération des réponses aux tickets (n8n, Zammad, Ollama, Telegram), fournissant une aide à l'agent qui examine, modifie et valide chaque réponse avant envoi. La seconde, plus longue, a été consacrée à l'agent vocal : mise en place de l'infrastructure (Asterisk, bridge.py, AudioSocket), intégration STT/TTS, débogage et robustesse (mémoire, latence, fiabilité), résolution de problèmes réseau et 3CX, puis ajout du message d'accueil et refonte du script de collecte.

1.6 Conclusion

Ce chapitre a présenté CYBERSHIELD TN, la problématique – une gestion manuelle mobilisant le temps des équipes SOC – et la solution envisagée, articulée autour des outils existants (3CX, Zammad) enrichis de reconnaissance vocale, de génération de langage et de synthèse vocale. Le chapitre suivant détaille la conception de cette solution.

Conception de la solution

2.1 Introduction

Ce chapitre détaille la conception de la solution : architecture globale des deux pipelines développés selon une démarche itérative, puis choix technologiques retenus pour chacun de leurs composants.

2.2 Architecture globale de la solution

2.2.1 Vue d'ensemble et démarche itérative

Le premier pipeline a porté sur la notification des alertes Zammad vers Telegram (priorités *high, urgent, critical*), enrichie d'une proposition de solution par Ollama et d'une validation humaine obligatoire (*human-in-the-loop*). Le second a étendu cette automatisation à l'appel téléphonique lui-même : interception, transcription, collecte d'informations et création automatique du ticket, sans intervention humaine pour cette qualification initiale. Les deux pipelines partagent n8n et Zammad.

2.2.2 Pipeline de notification, de triage et d'escalade des tickets (Telegram – Ollama – Zammad)

À la création d'un ticket, son contenu est transmis à Ollama, qui génère une proposition de solution destinée à assister l'agent. Cette proposition est soumise à un agent via Telegram, qui la consulte, la modifie si besoin, et l'approuve explicitement avant tout envoi.

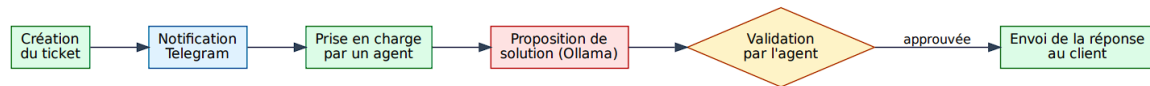


FIGURE 2.1 – Pipeline de génération assistée et de réponse aux tickets

Si aucun agent ne répond ou n'est disponible, le ticket est automatiquement réassigné (figure 2.1), garantissant qu'aucune demande ne reste sans réponse.

2.2.3 Pipeline d'appel en direct (3CX – Asterisk – bridge RTP – STT – bot – TTS)

L'appel est routé par 3CX vers Asterisk, qui établit une connexion audio temps réel avec une passerelle dédiée (`bridge.py`) via `AudioSocket`. Cette passerelle bufferise l'audio par tours de parole, le transmet à `n8n` pour transcription (`faster-whisper`), fait progresser un script de collecte via un LLM local, puis restitue la réponse (`edge-tts`) via la même passerelle. Le ticket est créé automatiquement une fois toutes les informations recueillies.

2.3 Choix technologiques

Ces technologies ont été retenues selon deux critères communs : la maîtrise des données traitées et la facilité d'intégration dans une architecture par webhooks/HTTP.

2.3.1 n8n (orchestration des workflows)

Plateforme open source auto-hébergée, garantissant la confidentialité des données, avec des workflows visuels facilitant l'itération et le débogage.

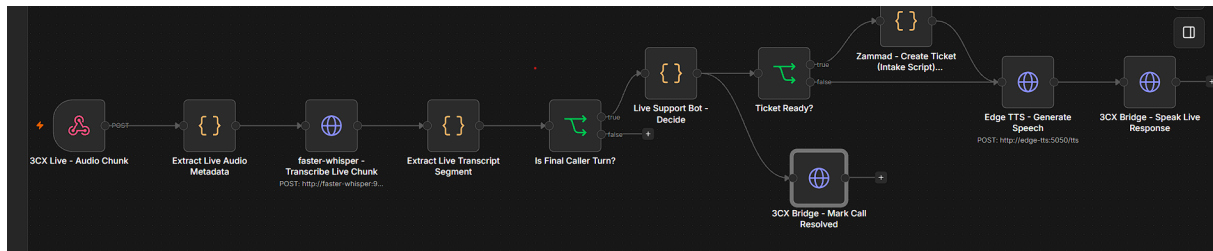


FIGURE 2.2 – Extrait du workflow n8n du pipeline d’appel en direct

2.3.2 Ollama (LLM local)

Exécution locale du modèle de langage, pour la génération de solutions (triage) comme pour la collecte d’informations (appel), garantissant la confidentialité des données clients et l’indépendance vis-à-vis d’un service cloud, au prix d’un temps de réponse dépendant des ressources locales.

2.3.3 Redis (file d’attente et gestion de la concurrence)

Introduit lors de la migration de n8n d’un conteneur unique vers un mode file d’attente (3 workers), pour absorber les alertes arrivant en rafale.

```
services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n
    restart: always
    ports:
      - "5678:5678"
    environment:
      - N8N_PORT=5678
      - NODE_ENV=production
      - WEBHOOK_URL=http://sandpaper-steerable-blandness.ngrok-free.dev/
      - N8N_SECURE_COOKIE=false
      - NODES_EXCLUDE=[]
      - GENERIC_TIMEZONE=Africa/Tunis
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=redis
      - QUEUE_BULL_REDIS_PORT=6379
      - N8N_RESTRICT_FILE_ACCESS_TO=/data/shared
      - BRIDGE_URL=https://retrace-font-implicit.ngrok-free.dev
    volumes:
      - ~/.n8n:/home/node/.n8n
      - ./shared:/data/shared
      - ./home/node/soc-project
    depends_on:
      - redis

  n8n-worker:
    image: n8nio/n8n:latest
    restart: always
    command: worker
    environment:
      - N8N_PROTOCOL=http
      - NODE_ENV=production
      - GENERIC_TIMEZONE=Africa/Tunis
      - EXECUTIONS_MODE=queue
      - QUEUE_BULL_REDIS_HOST=redis
      - QUEUE_BULL_REDIS_PORT=6379
```

FIGURE 2.3 – Configuration Docker Compose de l’architecture n8n en mode queue avec Redis

Cette migration a aussi résolu une race condition : le suivi des tickets pris en charge, auparavant local à chaque worker (\$vars), a été déplacé vers un état partagé (\$getWorkflowStaticData('global')), évitant qu’un même ticket soit assigné à deux agents simultanément.

2.3.4 Zammad (gestion des tickets)

Conservé comme système central existant, interfacé via son API REST pour la création, mise à jour et réponse aux tickets depuis n8n.

2.3.5 Telegram (notification et validation par un agent)

Canal de notification et de validation choisi pour la simplicité de son API et son usage déjà répandu, mettant en œuvre le principe *human-in-the-loop* : toute proposition Ollama doit être explicitement validée avant envoi au client.

2.3.6 faster-whisper (reconnaissance vocale)

Moteur STT local, rapide et adapté au français, exposant une API HTTP simple.

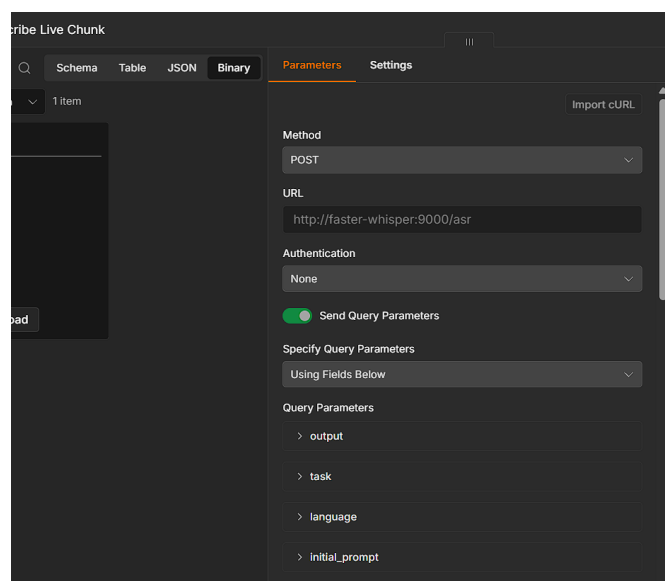


FIGURE 2.4 – Configuration de l'appel HTTP vers le service faster-whisper depuis n8n

2.3.7 edge-tts (synthèse vocale)

Service gratuit à la qualité vocale naturelle en français, retournant un fichier MP3 à convertir avant injection dans l'appel (détail au chapitre 3).

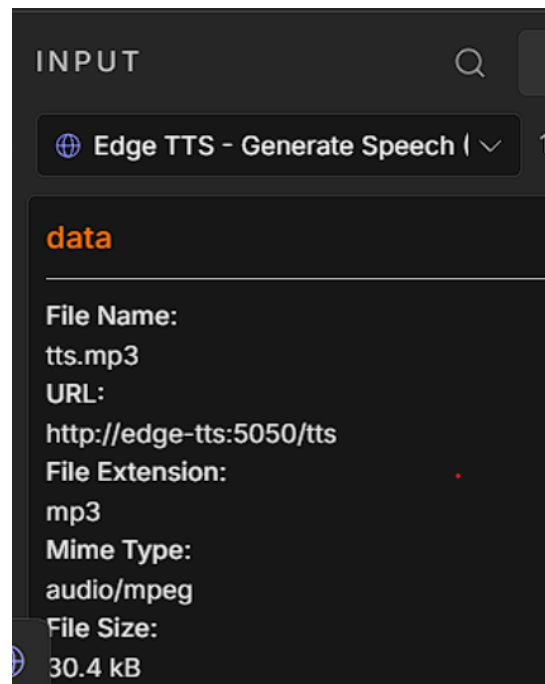


FIGURE 2.5 – Sortie du nœud Edge TTS - Generate Speech, montrant l’audio MP3 généré à partir du texte du bot

2.4 Conclusion

L’architecture des deux pipelines – triage/escalade des tickets et appel en direct – ainsi que les technologies retenues (n8n, Ollama, Redis, Zammad, Telegram, faster-whisper, edge-tts) ont été présentées, guidées par un principe constant de validation humaine. Le chapitre suivant détaille leur réalisation concrète.

Chapitre

3

Réalisation et tests

3.1 Introduction

Ce chapitre présente la réalisation concrète des deux pipelines, les difficultés rencontrées et les résultats des tests menés.

3.2 Environnement de travail

3.2.1 Outils et technologies (Docker, WSL2, VMware, ngrok)

Le développement a été réparti sur plusieurs machines : une VM VMware pour Asterisk, un environnement WSL2 (Docker) pour les services applicatifs (n8n, faster-whisper, edge-tts, Ollama, Redis), orchestrés via un `docker-compose.yml` unique. Des tunnels ngrok ont été nécessaires pour relier ces environnements réseau distincts, avant qu'une reconfiguration réseau ne permette une communication plus directe.

3.3 Orchestration des notifications et validation des réponses via Telegram

Ce pipeline assure la gestion automatisée des tickets prioritaires (*high*, *urgent*, *critical*) depuis leur création jusqu'à la validation d'une solution par un agent support.

3.3.1 Identification de l'agent et notification initiale

À la création d'un ticket dans Zammad, le pipeline identifie le propriétaire du ticket (`owner_id`) afin d'adresser la notification directement à l'agent concerné, évitant ainsi de solliciter inutilement l'ensemble de l'équipe.

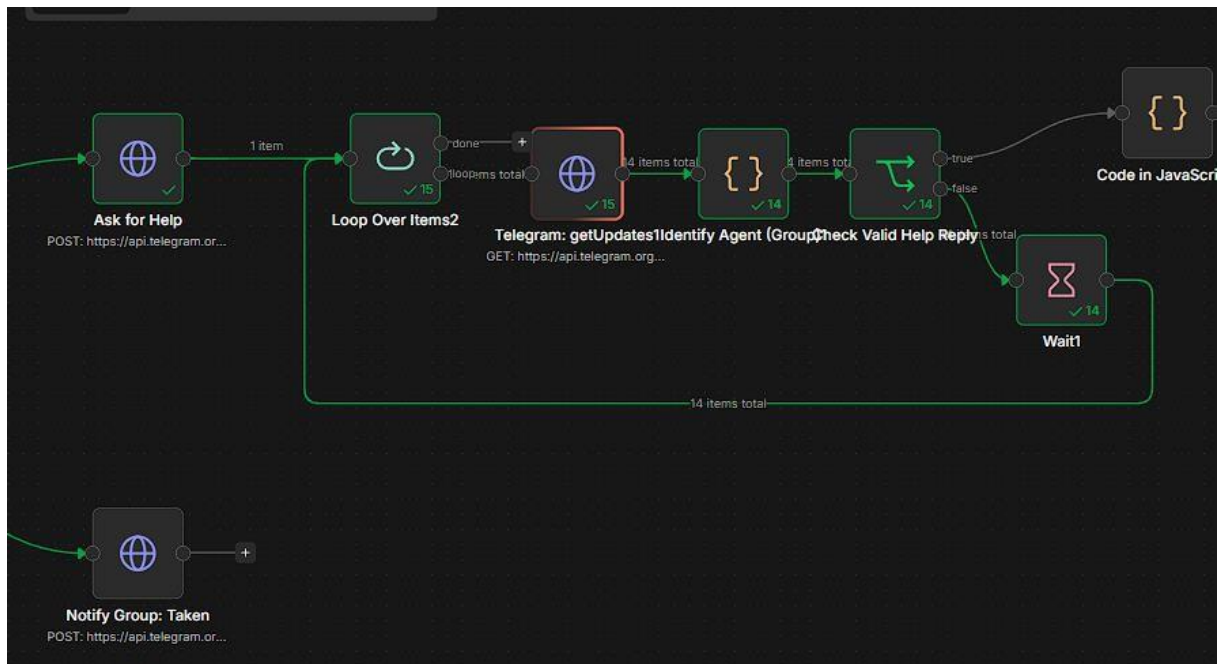


FIGURE 3.1 – Résolution de l'agent et notification du groupe lors de la prise en charge du ticket

L'agent reçoit sur Telegram une notification contenant les informations clés du ticket (numéro, sujet, client, résumé initial) assortie de boutons d'action interactifs pour prendre en charge le ticket ou le réassigner.



FIGURE 3.2 – Notification Telegram reçue lors de la création d’un ticket

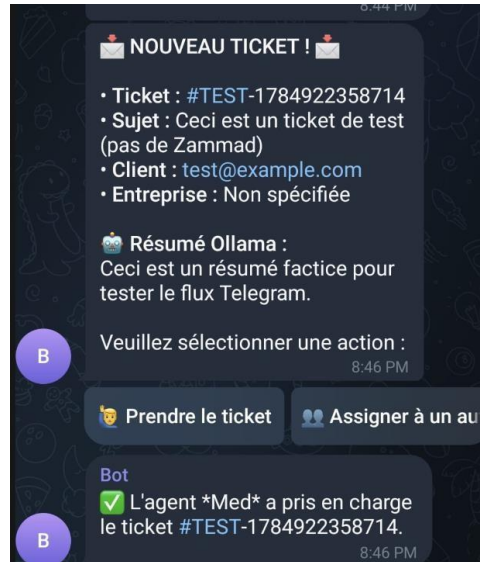


FIGURE 3.3 – Prise en charge du ticket par un agent

3.3.2 Sondage, relance et gestion de la disponibilité

Afin de garantir le traitement rapide des demandes, un mécanisme de sondage périodique (Telegram: `getUpdates`) écoute en permanence les réponses des agents.

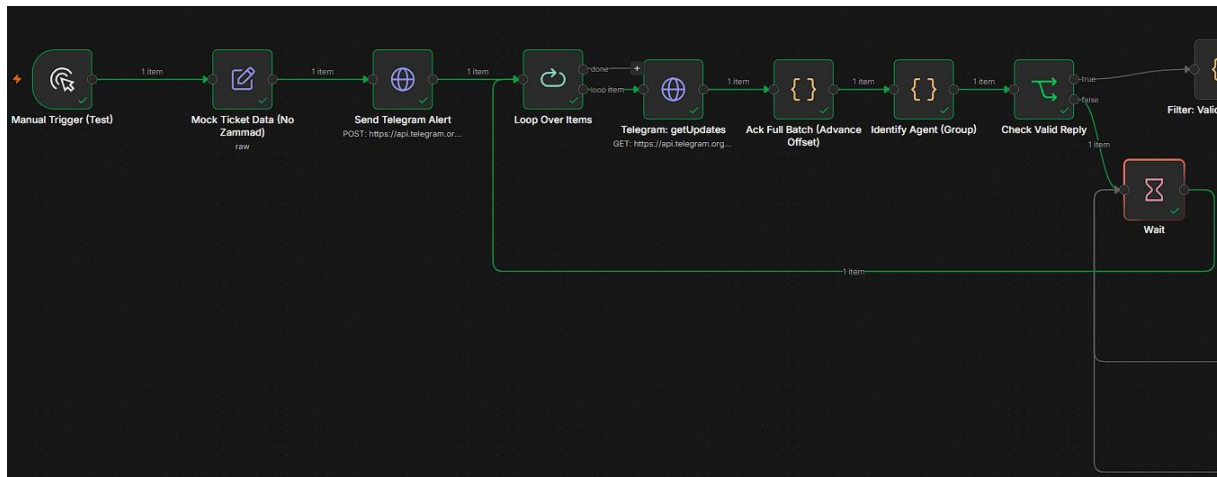


FIGURE 3.4 – Boucle de sondage Telegram en attente d’une réponse de l’agent

En cas d’absence de réponse dans les délais impartis ou lors d’une demande de réassignation explicite, le système déclenche une relance unique vers le groupe. Un nœud de garde (Guard: Notify Not Taken Once) ainsi que le suivi d’état centralisé sur Redis empêchent les notifications répétitives et garantissent qu’un ticket ne reste jamais non assigné ou attribué en double.

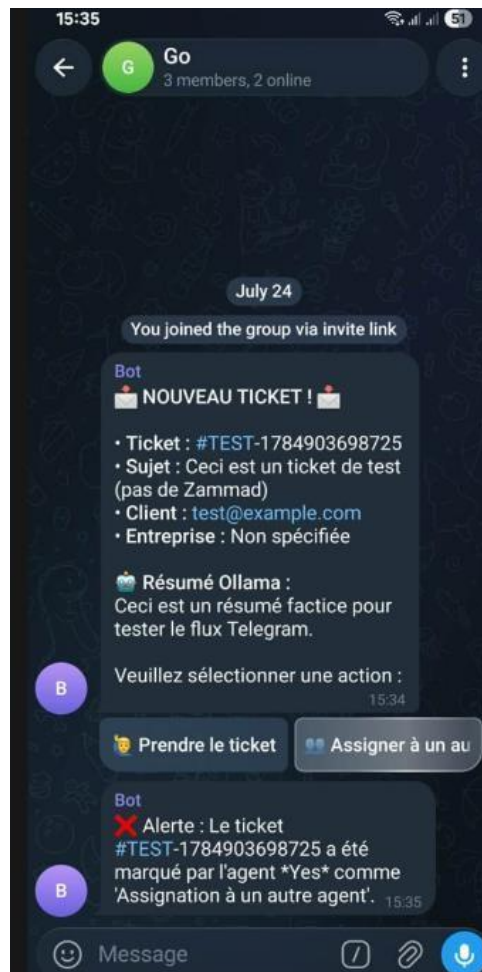


FIGURE 3.5 – Mécanisme de relance unique et de réassignation en cas d’absence de réponse

3.3.3 Génération automatique de propositions de solution par l’IA (Ollama)

Une fois le ticket pris en charge, son contenu est transmis au modèle LLM local (Ollama) via un prompt structuré. Ollama génère alors une proposition de résolution assistée et la transmet à l’agent sur Telegram pour évaluation.

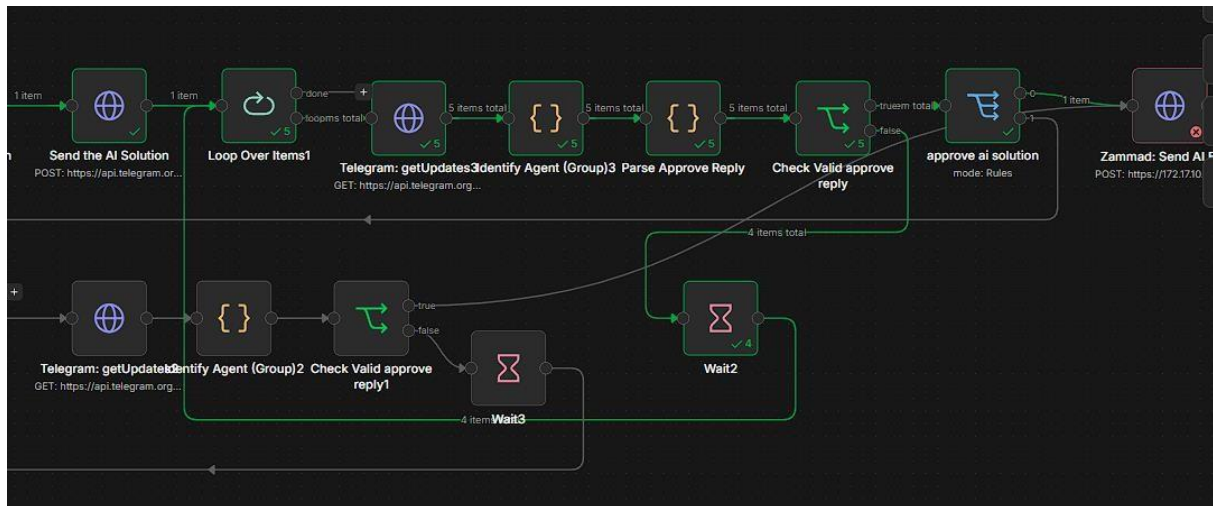


FIGURE 3.6 – Workflow n8n de génération et de validation de la solution IA

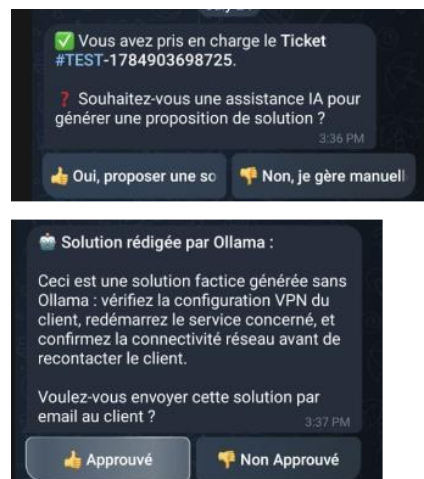


FIGURE 3.7 – Sollicitation de l’assistance IA par l’agent, puis proposition de solution générée par Ollama

3.3.4 Validation de la réponse et mécanisme *Force Reply*

Pour finaliser le traitement, l’agent peut soit valider la proposition de l’IA, soit fournir une réponse manuscrite personnalisée. Pour éviter tout risque de confusion lors du traitement simultané de plusieurs tickets, le mécanisme *Force Reply* de Telegram est utilisé afin d’associer sans ambiguïté la réponse de l’agent au ticket correspondant.



FIGURE 3.8 – Réponse de l’agent rattachée au ticket via le mécanisme Force Reply

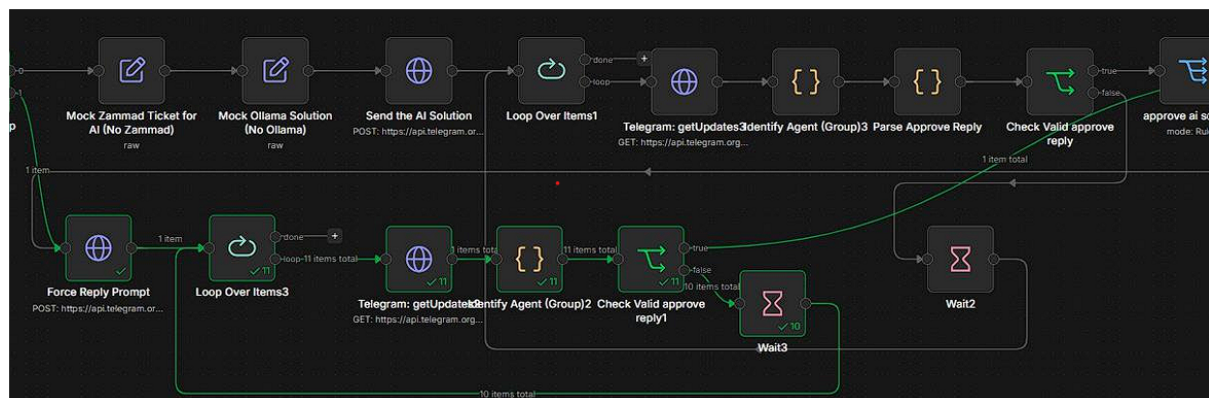


FIGURE 3.9 – Workflow n8n gérant l’attente et la validation d’une réponse manuscrite via Force Reply

Une fois la réponse validée par l’agent, le workflow met à jour l’état du ticket dans Zammad et transmet la solution finale au client.

3.4 Mise en œuvre du pipeline d’appel en direct

3.4.1 Transfert de l’appel du client vers l’agent IA

Avant que le pipeline de traitement vocal ne puisse intervenir, l’appel doit au préalable être acheminé depuis le standard 3CX jusqu’à l’extension virtuelle prise en charge par Asterisk. Ce

parcours technique, bien qu'invisible pour l'appelant, repose sur une succession précise d'étapes allant de la simple règle de renvoi jusqu'à l'ouverture effective du canal audio.

Sur l'extension cible, une règle de renvoi – ou, en phase de test, une règle sortante dédiée déclenchée par le préfixe 9101 – redirige l'appel vers l'extension associée à l'agent IA. Le trunk PJSIP configuré côté Asterisk reçoit alors cet appel ; le contexte de numérotation correspondant exécute successivement la commande `Answer()`, puis `AudioSocket(UUID, 127.0.0.1:8091)`, établissant ainsi la connexion avec la passerelle `bridge.py` qui prendra en charge la suite de l'échange.

The screenshot shows the 'Edit: 100_to_101' configuration page in 3CX. The 'Rule Name' is '100_to_101'. Under 'Calls to numbers starting with prefix', the value '9101' is entered. There are also fields for 'Calls from extension(s)' and 'Calls to numbers with a length of'. The 'Calls from Departments' section has a dropdown menu labeled 'Select departments...'. The 'Make outbound calls on' section contains a table for route configuration. The first route is named 'stage' and has 'Strip digits' set to 'Strip the first 1 digit(s)'. There are also fields for 'Prepend' and 'Outbound caller ID'.

FIGURE 3.10 – Règle de renvoi configurée dans 3CX (extension 100 vers l'extension IA via le préfixe 9101)

Dès que la connexion `AudioSocket` est établie, `bridge.py` lit l'identifiant unique (UUID) de l'appel et notifie sans délai n8n de son démarrage, via le webhook `Call Started`, en y joignant les métadonnées disponibles : numéro de l'appelant, DID composé et éventuelle saisie DTMF.

```
asterisk1@asterisk1-virtual-machine:~/3cx-bridge$ python3 bridge.py
[bridge] Registration HTTP server listening on :8090
AudioSocket server listening on 0.0.0.0:8091
[audiosocket] connection accepted from ('127.0.0.1', 36800)
[audiosocket] packet type=1 length=16
[audiosocket] UUID set: 1141fb32a1f411f1b48fc545cfd90e6d
[audiosocket] packet type=16 length=320
[bridge] Call 1141fb32a1f411f1b48fc545cfd90e6d audio stream started, metadata={}
[bridge] call-start POSTed, status=200
```

FIGURE 3.11 – Journal de `bridge.py` confirmant l'acceptation de la connexion `AudioSocket` et l'envoi du webhook `Call Started`

Le bon acheminement de l'appel peut être vérifié a posteriori dans l'historique des appels de 3CX, qui retrace fidèlement le chemin suivi – depuis la règle de renvoi initialement déclenchée jusqu'au trunk effectivement emprunté pour joindre Asterisk.

Call Time	Call ID	Caller	Callee	Direction	Status	Sentiment	Ringing	Talking	Cost	Call Activity Details
08/27/2026 11:38:43 AM	000-005	Med Omezzine (100)	101	Outbound	Answered	-	00:00:00	00:00:36	0.62	Dialed: (9101) → Via rule: 100_to_101 (9101) → Via trunk: stage (101) → Ended by Med Omezzine (100)
08/27/2026 11:37:57 AM	000-004	Med Omezzine (100)	101	Outbound	Answered	-	00:00:00	00:00:15	0.26	Dialed: (9101) → Via rule: 100_to_101 (9101) → Via trunk: stage (101) → Ended by Med Omezzine (100)

FIGURE 3.12 – Historique d'appels 3CX confirmant le routage effectif vers l'extension IA

3.4.2 Script de collecte d'informations (Live Support Bot - Decide)

Une fois la transcription d'un tour de parole obtenue, celle-ci est transmise au nœud Live Support Bot - Decide, cœur logique de l'agent vocal. Ce nœud implémente une machine à états qui guide progressivement l'appelant à travers un script structuré de collecte d'informations : nom et prénom, description du problème, confirmation du problème reformulé, confirmation du nom, adresse email, puis nom de la société. L'état courant de la conversation (step) est conservé pour chaque appel entre deux tours de parole, ce qui permet au bot de reprendre précisément là où l'échange s'était arrêté, sans jamais perdre le fil de la collecte en cours.

À chaque étape de confirmation, la réponse du client est comparée à deux listes de mots-clés – l'une associée à une acceptation (*oui, exact, voila...*), l'autre à un refus (*non, pas exact...*) – ce qui permet de déterminer simplement si l'information proposée doit être validée ou redemandée.

```
const confirmWords = ["oui", "ouais", "exact", "correct", "c est ca", "c'est ca", "c est bon", "cest bon", "voila"];
const denyWords = ["non", "pas exact", "incorrect", "c est pas ca", "c'est pas ca"];

function isConfirm(t) { return confirmWords.some(w => t.includes(w)); }
function isDeny(t) { return denyWords.some(w => t.includes(w)); }
```

FIGURE 3.13 – Détection des réponses de confirmation et de refus du client

La progression à travers les différentes étapes du script est quant à elle implémentée sous la forme d'une instruction switch portant sur l'état courant de l'appel, chaque cas correspondant à une étape précise de la collecte.

```
switch (state.step) {
  case "ask_name":
    if (isGarbage(message)) {
      bot_response = "Desole, je n'ai pas bien entendu. Pouvez-vous repeter votre nom et prenom ?";
      break;
    }
    state.name = message;
    state.step = "ask_problem";
    bot_response = "Merci. Quel est votre probleme ?";
    break;

  case "ask_problem":
    if (isGarbage(message)) {
      bot_response = "Desole, je n'ai pas bien entendu. Pouvez-vous repeter votre probleme ?";
      break;
    }
    state.problem = message;
    state.step = "confirm_problem";
    bot_response = "Si je comprends bien, votre probleme est : " + message + ". Est-ce exact ?";
    break;

  case "confirm_problem":

```

FIGURE 3.14 – Machine à états du script de collecte, avec validation des réponses via isGarbage()

Comme le montre la figure 3.14, chaque étape de collecte débute systématiquement par une vérification de la validité de la réponse transcrite, à travers une fonction dédiée, `isGarbage()`. L'ajout de cette validation s'est imposé à la suite d'un problème observé en conditions réelles : le modèle `faster-whisper` utilisé, dans sa variante *base*, rencontre des difficultés significatives pour transcrire correctement des informations isolées – noms propres, adresses email, noms de société – là où les phrases complètes bénéficient, elles, du contexte grammatical pour guider l'interprétation du modèle.

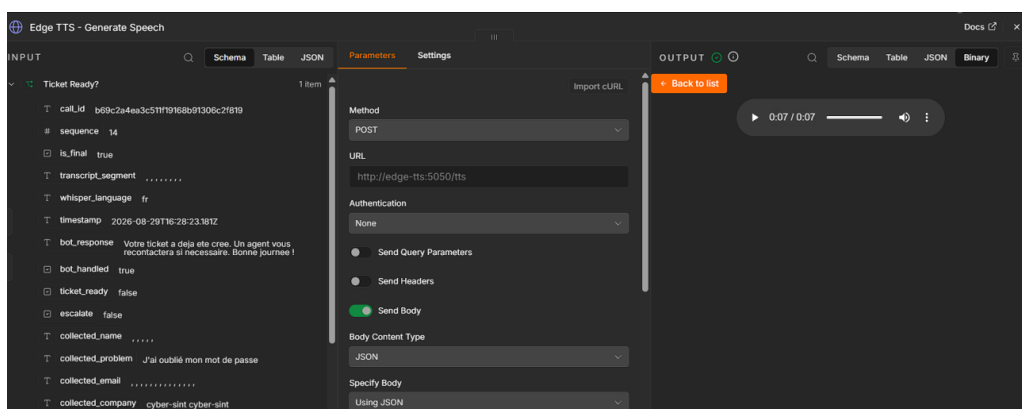


FIGURE 3.15 – Exemple réel de champs collectés corrompus par une transcription défaillante (nom et email vides, société mal transcrite)

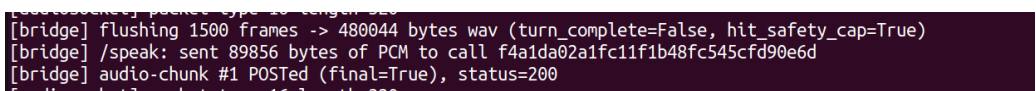
Plusieurs cas concrets illustrent cette limitation lors des tests menés : le nom et prénom « Mohamed Omezzine » a ainsi été transcrit sous la forme d'une simple suite de virgules, une adresse email est parfois retournée vide ou fragmentée, tandis que le nom de société « CYBERSHIELD » a été transcrit « cyber-sint cyber-sint » (figure 3.15). Cette limitation, inhérente aux modèles Whisper de petite taille appliqués à de l'audio téléphonique

sous-échantillonné (8 kHz), a directement motivé l'introduction de la fonction `isGarbage()` : plutôt que d'accepter telle quelle une transcription manifestement inexploitable, le bot redemande poliment

l'information au client, évitant ainsi de transmettre des données corrompues jusqu'au ticket final.

3.4.3 Boucle jusqu'à la fin de l'appel et création du ticket

Le cycle décrit précédemment se répète naturellement à chaque tour de parole, jusqu'à ce que l'état de la conversation atteigne `done`, ou jusqu'au raccrochage de l'appelant. Les journaux de `bridge.py` permettent de suivre ce cycle pas à pas : chaque tour détecté est bufferisé, converti en fichier WAV, transmis à n8n pour traitement, avant que la réponse vocale correspondante ne soit aussitôt réinjectée dans l'appel en cours.



```
[bridge] flushing 1500 frames -> 480044 bytes wav (turn_complete=False, hit_safety_cap=True)
[bridge] /speak: sent 89856 bytes of PCM to call f4a1da02a1fc11f1b48fc545cfd90e6d
[bridge] audio-chunk #1 POSTed (final=True), status=200
```

FIGURE 3.16 – Journal de `bridge.py` montrant un cycle complet de traitement d'un tour de parole

Une fois l'ensemble des informations collectées, le nœud `Ticket Ready?` vérifie que l'état de la conversation est bien passé à `done` ; le cas échéant, un ticket est créé automatiquement dans Zammad avant que la confirmation finale ne soit synthétisée et restituée au client. En parallèle de cette vérification, le nœud `3CX Bridge - Mark Call Resolved` notifie la passerelle de l'état de résolution de l'appel, une information qui sera réutilisée à la toute fin de l'appel, au niveau du webhook `Call Ended`, pour déterminer si un ticket doit encore être créé ou non.

3.4.4 Workflow n8n « Call to Ticket »

Les sous-sections précédentes ont détaillé isolément chacun des composants intervenant dans le traitement d'un appel : la passerelle `bridge.py`, les microservices de reconnaissance et de synthèse vocale, ainsi que la machine à états pilotant la collecte d'informations. Cette dernière sous-section propose une vue d'ensemble du workflow n8n qui orchestre l'intégralité

de ce traitement, déjà illustré à la figure 2.2 du chapitre 2, en reprenant la chaîne complète des nœuds impliqués ainsi que la logique de branchement qui la structure.

La chaîne de nœuds concernée est la suivante : 3CX Live - Audio Chunk (webhook recevant l'audio depuis `bridge.py`), Extract Live Audio Metadata (extraction du `call_id`, du numéro de séquence et du statut `is_final`), faster-whisper - Transcribe Live Chunk, Extract Live Transcript Segment, Is Final Caller Turn? (détermine si le tour de parole doit être traité ou si l'audio doit encore être accumulé), Live Support Bot - Decide, qui se branche ensuite en parallèle vers 3CX Bridge - Mark Call Resolved d'une part, et vers Ticket Ready? d'autre part, avant que le flux ne converge vers Edge TTS - Generate Speech puis 3CX Bridge - Speak Live Response.

Comme le montre la figure 2.2, la mise en œuvre de ce workflow peut se résumer en six étapes principales.

Étape 1 – Réception de l'audio : `bridge.py` bufferise l'audio de l'appelant par tours de parole et le transmet à n8n via un webhook, au format WAV.

Étape 2 – Transcription (STT) : le segment audio est envoyé à faster-whisper, qui retourne le texte transcrit correspondant, en français.

Étape 3 – Décision du bot : le texte transcrit fait progresser le script de collecte d'informations (nom, problème, confirmation, email, société) au sein du nœud Live Support Bot - Decide, qui déclenche ensuite deux branches exécutées en parallèle.

Étape 4 – Statut de l'appel et création du ticket : la première branche (Mark Call Resolved) notifie `bridge.py` de l'état de résolution de l'appel. La seconde (Ticket Ready?) vérifie si l'ensemble des informations a été collecté; si c'est le cas, le ticket est créé dans Zammad avant de poursuivre le flux, sinon celui-ci se dirige directement vers l'étape de synthèse vocale.

Étape 5 – Synthèse vocale (TTS) : la réponse du bot – qu'il s'agisse de la question suivante ou d'une confirmation – est transmise à edge-tts, qui génère le fichier audio correspondant au format MP3.

Étape 6 – Lecture dans l'appel : ce fichier audio est envoyé à `bridge.py`, converti en PCM 8 kHz, puis injecté directement dans l'appel en cours via `AudioSocket`, permettant à l'appelant d'entendre la réponse du bot en temps réel.

Ce découpage en étapes met en évidence la nature cyclique du workflow : chaque tour de parole de l'appelant déclenche l'intégralité de cette chaîne, jusqu'à ce que le script de collecte soit mené à son terme ou que l'appel prenne fin.

3.5 Difficultés rencontrées et solutions

Le développement de la solution, en particulier du pipeline d'appel en direct, s'est heurté à plusieurs difficultés techniques significatives. Cette section revient sur les principales d'entre elles, regroupées par catégorie, ainsi que sur les solutions mises en place pour y remédier.

3.5.1 Problèmes réseau entre environnements distribués

La répartition de la solution sur plusieurs machines – la machine virtuelle VMware hébergeant Asterisk, l'environnement WSL2 hébergeant les services applicatifs, et le serveur 3CX – a constitué la source la plus fréquente de difficultés rencontrées durant le stage. Parmi les problèmes rencontrés :

- des tunnels ngrok, utilisés pour exposer les services entre ces différents environnements réseau, expirant sans prévenir et provoquant des erreurs de type « connection aborted » ;
- une erreur de configuration dans le fichier `pjsip.conf` d'Asterisk (adresse IP du serveur 3CX incorrectement renseignée), empêchant Asterisk d'accepter les appels entrants.

Un problème réseau plus large, indépendant du pipeline vocal lui-même, a également été diagnostiqué et résolu durant le stage : plusieurs extensions 3CX (100, 101, 102, 801) apparaissaient comme « Not Registered ». Le diagnostic a révélé une panne du serveur DNS interne de l'entreprise : les requêtes `nslookup` vers le nom de domaine du serveur 3CX expiraient systématiquement, provoquant un repli vers un serveur DNS externe qui renvoyait l'adresse IP

publique du serveur au lieu de son adresse interne. Les appareils situés pourtant sur le même réseau local que le serveur 3CX se retrouvaient ainsi à tenter de le joindre via son IP publique, provoquant un échec de type NAT loopback.

En l'absence d'accès aux équipements réseau permettant de résoudre la cause racine, un contournement temporaire a été mis en place sur le poste concerné : l'ajout d'une entrée manuelle dans le fichier *hosts*, forçant la résolution du nom de domaine du serveur 3CX vers son adresse IP interne plutôt que publique.

```
PS C:\Users\medga> Add-Content -Path "C:\Windows\System32\drivers\etc\hosts" -Value "172.16.58.36 cybershield-stage.3cx.eu"
```

FIGURE 3.17 – Ajout d'une entrée manuelle dans le fichier hosts pour forcer la résolution vers l'adresse IP interne du serveur 3CX

Cette correction a permis de restaurer immédiatement l'enregistrement de l'extension 100 sur le client Web du PC.

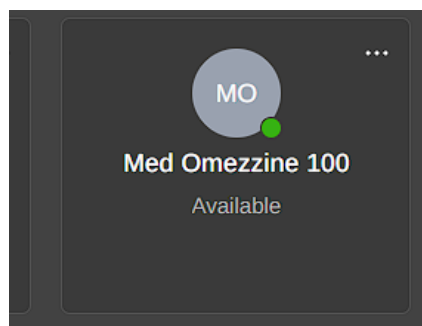


FIGURE 3.18 – Extension 100 enregistrée avec succès après correction DNS, passant au statut « Available »

Comme le montre la figure 3.18, l'extension est repassée à un statut opérationnel, confirmé par un appel de test réussi. Ce diagnostic, ainsi que le détail des étapes de résolution, a été documenté et transmis à l'équipe réseau de l'entreprise en vue d'une correction définitive – la restauration du serveur DNS interne – seule solution permettant un enregistrement automatique de l'ensemble des extensions sans intervention manuelle poste par poste.

3.5.2 Fiabilité de la passerelle bridge.py

Un bug significatif a été identifié dans la gestion des erreurs de `bridge.py` : l'échec ponctuel de l'envoi d'un seul chunk audio (par exemple lors d'un ralentissement temporaire

de `faster-whisper`) provoquait l'interruption complète de la fonction traitant l'appel, empêchant l'envoi du webhook `Call Ended` à `n8n`. Ce comportement bloquait silencieusement la création du ticket correspondant, sans qu'aucune erreur explicite ne soit visible côté utilisateur. La correction a consisté à envelopper chaque étape du traitement dans un bloc de gestion d'erreurs indépendant, garantissant que le webhook de fin d'appel soit systématiquement envoyé, quelle que soit l'issue des étapes précédentes.

Un second problème de fiabilité concernait la restitution de la voix du bot : les trames audio étaient initialement envoyées à Asterisk en rafale plutôt qu'à un rythme respectant leur durée réelle, provoquant des coupures ou des troncatures perceptibles de la réponse vocale. Ce problème a été corrigé en cadencant l'envoi des trames à 20 millisecondes d'intervalle, conformément au rythme attendu par le protocole `AudioSocket`.

3.5.3 Erreur de validation du workflow n8n

Une erreur de validation particulière a bloqué l'exécution complète du workflow principal pendant une phase du développement : la présence de nœuds `Respond to Webhook` non exploités, les déclencheurs correspondants étant configurés en mode de réponse immédiate. Bien que ces nœuds n'aient jamais été fonctionnellement nécessaires, leur simple présence a fini par constituer une erreur bloquante empêchant toute exécution du workflow, y compris sur des appels autrement valides.

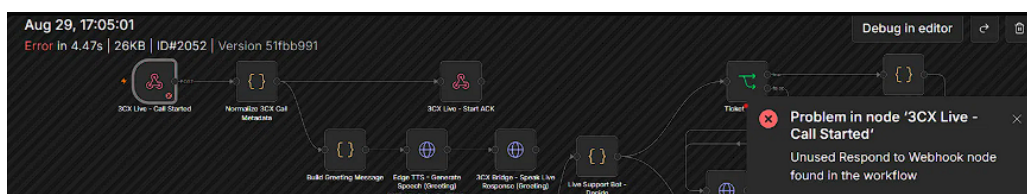


FIGURE 3.19 – Erreur de validation n8n bloquant l'exécution du workflow suite à la présence de nœuds `Respond to Webhook` non utilisés

La solution a consisté à supprimer purement et simplement ces nœuds devenus inutiles, rétablissant immédiatement l'exécution normale du workflow.

3.5.4 Latence perçue par l'appelant

Le seuil de silence utilisé par `bridge.py` pour détecter la fin d'un tour de parole était initialement fixé à 10 secondes, une valeur pensée par prudence mais qui s'est révélée excessive à l'usage : l'appelant devait attendre 10 secondes de silence complet avant que le bot ne commence à traiter sa réponse, ce qui rendait l'échange perceptiblement peu naturel et pouvait porter le temps d'exécution d'un tour complet à près de 20 secondes. Ce seuil a depuis été réduit à une valeur plus proche de 1 à 2 secondes, suffisante pour laisser à l'appelant le temps de terminer naturellement sa phrase, tout en rendant l'interaction sensiblement plus réactive.

3.5.5 Limites du modèle de reconnaissance vocale

Comme détaillé à la section 3.4.2, le modèle `faster-whisper` utilisé rencontre des difficultés notables pour transcrire correctement des informations isolées sans contexte grammatical – noms propres, adresses email, noms de société – une limitation inhérente aux modèles Whisper de petite taille appliqués à de l'audio téléphonique. Si l'ajout d'une validation des réponses (`isGarbage()`) a permis d'éviter la transmission de données manifestement corrompues, cette limitation reste une contrainte connue du système, qui pourrait être atténuée par l'usage d'un modèle plus volumineux ou par l'ajout d'un `initial_prompt` orienté vers ce type de contenu.

3.6 Conclusion

Ce chapitre a présenté la réalisation concrète des deux pipelines conçus au chapitre précédent : l'automatisation de la notification et de l'escalade des tickets de support, d'abord centrée sur la remontée des alertes prioritaires (*high, urgent, critical*) puis enrichie d'une proposition de solution par IA et d'une validation humaine obligatoire, ainsi que l'agent vocal capable de traiter les appels téléphoniques en direct. L'environnement de travail distribué sur plusieurs machines a été détaillé, tout comme la mise en œuvre technique de chaque composant – la passerelle

bridge.py, l'intégration des microservices de reconnaissance et de synthèse vocale, le script de collecte d'informations, ainsi que le workflow n8n orchestrant l'ensemble.

Les difficultés rencontrées durant le développement – contraintes de mémoire, instabilités réseau entre environnements distribués, bugs de fiabilité, limites du modèle de reconnaissance vocale – ont été présentées avec les solutions apportées à chacune d'entre elles. Les tests menés sur les deux pipelines ont permis de valider leur bon fonctionnement dans leurs principaux scénarios d'usage, tout en mettant en lumière certaines limites persistantes, notamment liées à la transcription d'informations isolées par le modèle de reconnaissance vocale.

Ce travail a permis de mettre en place une solution fonctionnelle et opérationnelle, respectant le principe de validation humaine ayant guidé l'ensemble du projet, et ouvre plusieurs perspectives d'amélioration – utilisation d'un modèle de transcription plus performant, réduction supplémentaire de la latence perçue, ou encore extension du script de collecte à d'autres types de demandes – qui pourront être explorées dans la continuité de ce stage.



CONCLUSION GÉNÉRALE

Ce stage d'été de deux mois, effectué au sein de CYBERSHIELD TN, a été l'occasion de concevoir et de mettre en œuvre une solution complète d'automatisation destinée à améliorer la gestion des appels et des tickets de support client. Parti d'un constat simple – une gestion largement manuelle des demandes clients, mobilisant un temps considérable pour les équipes du SOC – ce travail a permis d'aboutir à deux systèmes complémentaires : un pipeline de notification et d'escalade automatisée des tickets, d'abord centré sur la remontée des alertes prioritaires (*high, urgent, critical*) via Telegram, puis enrichi d'une proposition de solution par Ollama et d'une validation humaine obligatoire ; et un agent vocal capable de traiter des appels téléphoniques en direct, intégrant reconnaissance vocale, génération de langage et synthèse vocale au sein d'une architecture orchestrée par n8n.

La démarche suivie tout au long de ce stage a été itérative : chaque système a été construit progressivement, testé en conditions réelles, puis consolidé à mesure que des difficultés étaient identifiées – qu'il s'agisse de contraintes de ressources, d'instabilités réseau entre environnements distribués, de bugs de fiabilité, ou encore de limites inhérentes aux modèles d'intelligence artificielle utilisés. Cette approche a permis de garder, à chaque étape, une solution fonctionnelle et testable, plutôt que de chercher à atteindre d'emblée un système parfait.

Au-delà de l'aspect technique, ce stage a été une réelle opportunité d'appliquer et d'approfondir des compétences variées : l'intégration de systèmes hétérogènes, le développement d'une passerelle réseau bas niveau en Python, la conception de workflows d'automatisation, ainsi qu'une sensibilisation concrète aux enjeux de fiabilité et de validation humaine lors de l'introduction de l'intelligence artificielle dans un processus métier. Le principe de validation

humaine obligatoire, présent dans les deux pipelines développés, a constitué un fil conducteur tout au long du projet : garantir qu’aucune décision automatisée ne puisse atteindre un client sans un contrôle explicite, qu’il s’agisse d’une réponse générée par un modèle de langage ou d’un ticket créé à l’issue d’un appel.

Plusieurs perspectives d’amélioration se dégagent naturellement de ce travail. Du côté du pipeline vocal, l’usage d’un modèle de reconnaissance vocale plus performant, ou l’ajout de mécanismes complémentaires de validation orale, permettrait de réduire davantage les erreurs de transcription observées sur les informations isolées. Le script de collecte pourrait également être étendu à d’autres types de demandes, au-delà du scénario de prise de ticket initial. Du côté du pipeline de triage, une intégration plus poussée avec les outils existants de l’entreprise, ainsi qu’un affinement continu des prompts soumis au modèle de langage, contribueraient à améliorer encore la pertinence des solutions proposées aux agents.

En définitive, ce stage a permis de livrer une solution concrète et opérationnelle, tout en offrant une expérience professionnelle formatrice sur la conception de systèmes d’automatisation fiables intégrant l’intelligence artificielle, dans un contexte exigeant en matière de sécurité et de confiance.



Bibliographie

- [1] **n8n**. n8n Documentation [**en ligne**]. Disponible sur : <https://docs.n8n.io/>
- [2] **Ollama**. Ollama Documentation [**en ligne**]. Disponible sur : <https://ollama.com/>
- [3] **SYSTRAN**. faster-whisper : Faster Whisper transcription with CTranslate2 [**en ligne**].
Disponible sur : <https://github.com/SYSTRAN/faster-whisper>
- [4] **rany2**. edge-tts : Use Microsoft Edge's online text-to-speech service from Python [**en ligne**]. Disponible sur : <https://github.com/rany2/edge-tts>
- [5] **Zammad**. Zammad Documentation [**en ligne**]. Disponible sur : <https://docs.zammad.org/>
- [6] **Telegram**. Telegram Bot API [**en ligne**]. Disponible sur : <https://core.telegram.org/bots/api>
- [7] **Redis**. Redis Documentation [**en ligne**]. Disponible sur : <https://redis.io/docs/>
- [8] **Asterisk**. Asterisk Documentation [**en ligne**]. Disponible sur : <https://docs.asterisk.org/>
- [9] **Asterisk**. AudioSocket – Channel Driver Documentation [**en ligne**]. Disponible sur :
<https://docs.asterisk.org/Configuration/Channel-Drivers/AudioSocket/>
- [10] **3CX**. 3CX Documentation [**en ligne**]. Disponible sur : <https://www.3cx.com/docs/>
- [11] **Docker**. Docker Documentation [**en ligne**]. Disponible sur : <https://docs.docker.com/>